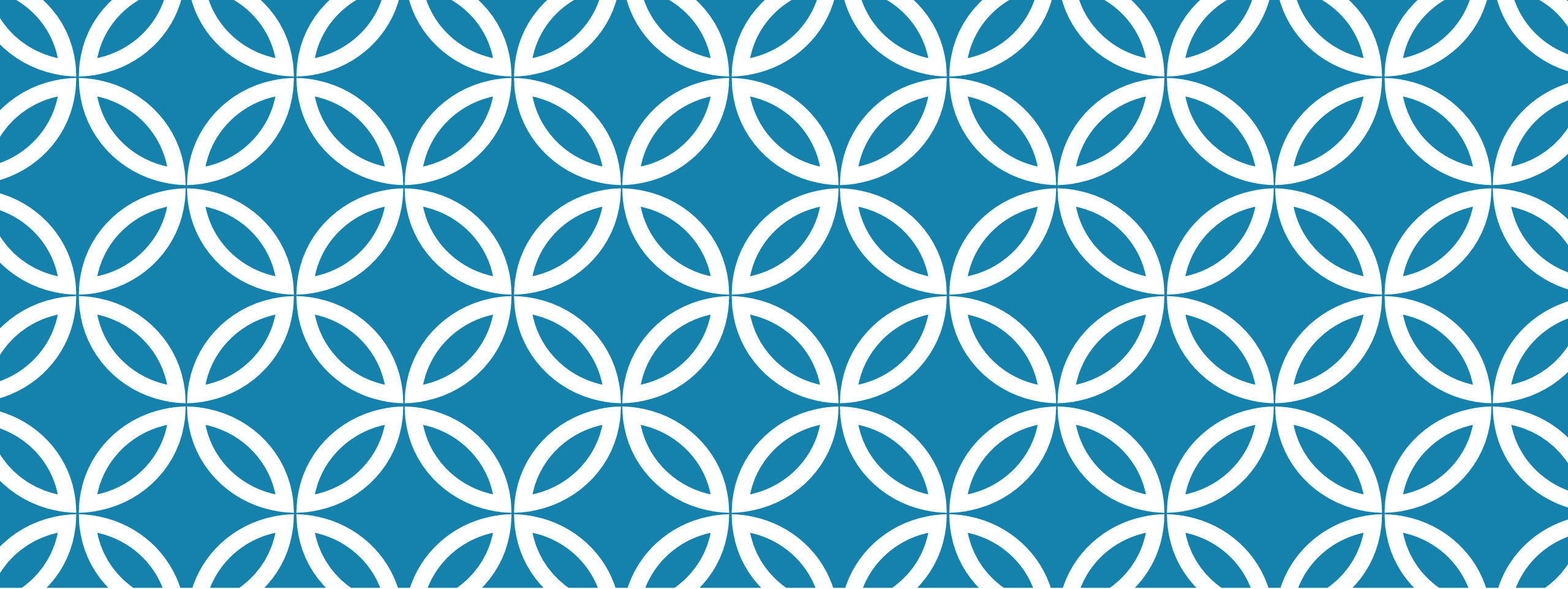




STACK DAN QUEUE DENGAN ARRAY

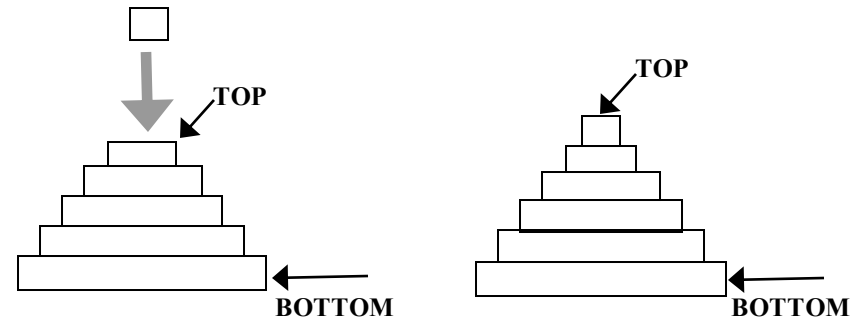
ALGORITMA DAN STRUKTUR DATA



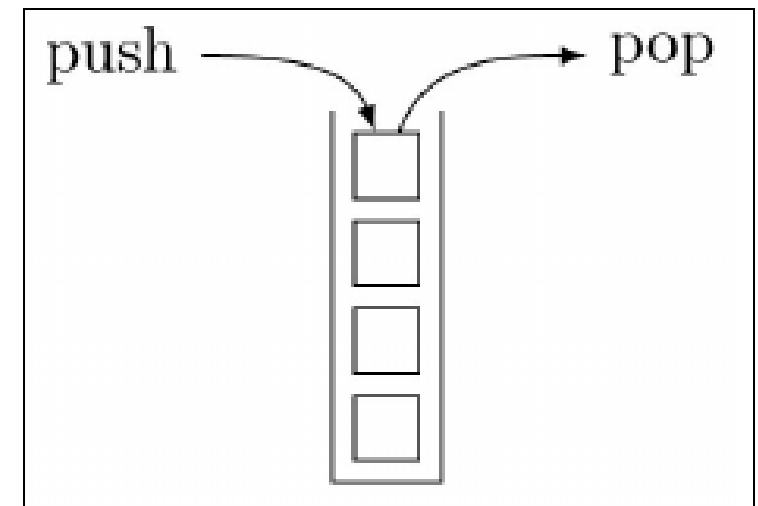
Institut Teknologi Sumatera

STACK

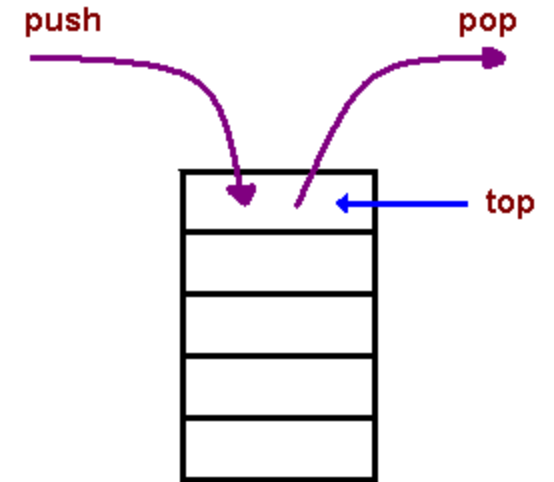
PENDAHULUAN



- Struktur data merupakan langkah dalam penyimpanan data bersama dengan kumpulan operasi untuk manipulasi data tersebut.
- Struktur data sederhana yang dikenal dengan stack, yang merupakan bentuk abstraksi dari tumpukan secara vertikal pada obyek secara fisik.



PENDAHULUAN



- Stack merupakan list linier yang:
 - ✓ Dikenali elemen puncaknya (TOP)
 - ✓ Aturan penyisipan dan penghapusan elemennya tertentu:
 - ❖ Penyisipan selalu dilakukan "di atas" TOP
 - ❖ Penghapusan selalu dilakukan pada TOP
- TOP adalah satu-satunya alamat tempat terjadi operasi
- Last-In-First-Out (LIFO)

DEFINISI FUNGSIONAL

Fungsi	Keterangan
void CreateEmpty()	Membuat sebuah stack kosong
boolean IsEmpty()	Menguji apakah stack dalam kondisi kosong, true jika stack kosong, dan false jika tidak
boolean IsFull()	Menguji apakah stack dalam kondisi penuh, true jika stack penuh, dan false jika tidak
void Push()	Menambahkan sebuah elemen, sehingga kondisi TOP saat ini akan berubah
void Pop()	Mengambil sebuah elemen yang merupakan TOP, dan TOP akan berubah

DEFINISI FUNGSIONAL

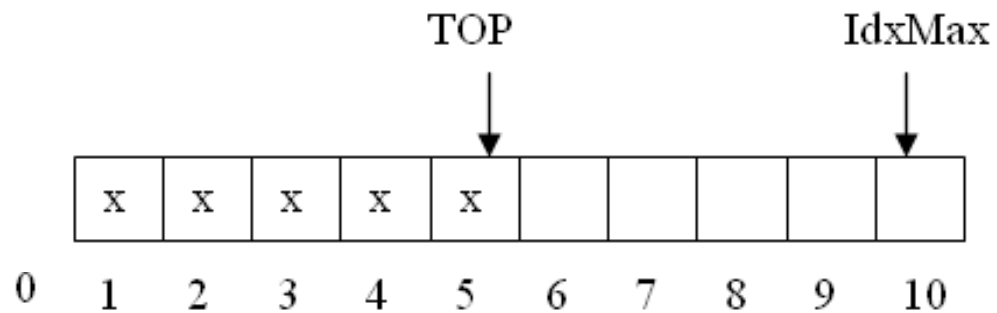
- Definisi Selektor:
 - ✓ Jika **S** adalah sebuah Stack, maka
 - ❖ **Top(S)** adalah alamat elemen TOP, di mana operasi penyisipan/penghapusan dilakukan
 - ❖ **InfoTop(S)** adalah informasi yang disimpan pada Top(S)
- Definisi **Stack kosong** adalah Stack dengan **Top(S)=NULL** (tidak terdefinisi)

REPRESENTASI STACK DENGAN ARRAY

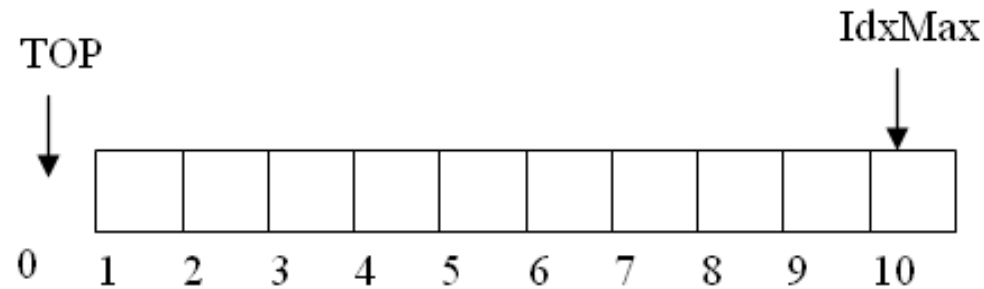
- Representasi array sangat tepat untuk mewakili Stack
 - ✓ Pada Stack : operasi penambahan dan pengurangan hanya dilakukan di salah satu “ujung” array
 - ✓ Pada array: operasi boleh di manapun.

REPRESENTASI STACK DENGAN ARRAY

- Ilustrasi Stack tidak kosong, dengan 5 elemen:



- Ilustrasi Stack kosong



IMPLEMENTASI STACK DENGAN ARRAY

```
#define Nil 0
#define MaxEl 10

typedef int infotype, address;
typedef struct TEltList{
    infotype Data[MaxEl + 1];
    address TOP;
} stack;

#define TOP(S)      (S).TOP
#define InfoTop(S)  (S).Data[(S).TOP]
```

IMPLEMENTASI STACK DENGAN ARRAY

```
void CreateEmpty(stack *S) {  
    TOP(*S) = Nil;  
}
```

```
bool IsEmpty(stack S) {  
    return (TOP(S) == Nil);  
}
```

```
bool IsFull(stack S) {  
    return (TOP(S) == MaxEl);  
}
```

IMPLEMENTASI STACK DENGAN ARRAY

```
void Push(stack *S, infotype x) {  
    if (!IsFull(*S)) {  
        TOP(*S)++;  
  
        InfoTop(*S) = x;  
    } else  
        cout << "Stack Penuh" << endl;  
}
```

IMPLEMENTASI STACK DENGAN ARRAY

```
void Pop(stack *S, infotype *x) {  
    if (!IsEmpty(*S)) {  
        *x = InfoTop(*S);  
  
        TOP(*S) --;  
    } else  
        cout << "Stack Kosong" << endl;  
}
```

IMPLEMENTASI STACK DENGAN ARRAY

```
int main() {
    stack DataTest;
    infotype DataHapus;

    CreateEmpty(&DataTest);

    cout << "Push(3) " << endl; Push(&DataTest, 3);
    .....
    .....
    cout << "Pop() "; Pop(&DataTest, &DataHapus);
    cout << "Hapus " << DataHapus << endl;

    cout << endl << "Isi Stack" << endl;

    while (!IsEmpty(DataTest)) {
        Pop(&DataTest, &DataHapus);

        cout << DataHapus << endl;
    }
}
```

QUEUE

PENDAHULUAN



- QUEUE adalah list linier yang:
 - ✓ Dikenali elemen pertama (HEAD) dan elemen terakhirnya (TAIL)
 - ✓ Aturan penyisipan dan penghapusan elemennya didefinisikan sebagai berikut:
 - Penyisipan selalu dilakukan setelah elemen terakhir
 - Penghapusan selalu dilakukan pada elemen pertama
 - ✓ Satu elemen dengan yang lain dapat diakses melalui informasi NEXT

PENDAHULUAN

- Elemen Queue tersusun secara FIFO (First In First Out)
- Pemakaian queue:
 - ✓ Antrian job yang harus ditangani oleh sistem operasi (job scheduling)
 - ✓ Antrian dalam dunia nyata

DEFINISI FUNGSIONAL

Fungsi	Keterangan
void CreateEmpty()	Membuat sebuah antrian kosong
boolean IsEmpty()	Menguji apakah antrian dalam kondisi kosong, true jika antrian kosong, dan false jika tidak
boolean IsFull()	Menguji apakah antrian dalam kondisi penuh, true jika antrian penuh, dan false jika tidak
void Add()	Menambahkan sebuah elemen setelah ekor (elemen terakhir) dari antrian
void Dell()	Menghapus kepala (elemen pertama) dari antrian
int NbElmt()	Memberikan info banyaknya elemen antrian

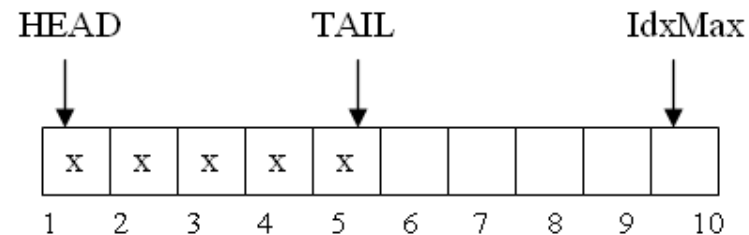
REPRESENTASI QUEUE DENGAN ARRAY

- Memori tempat penyimpan elemen adalah sebuah array dengan indeks 1.. IdxMax.
- IdxMax dapat juga “dipetakan” ke kapasitas Queue
- Representasi field Next: Jika i adalah “address” sebuah elemen, maka suksesor i adalah Next dari elemen Queue.

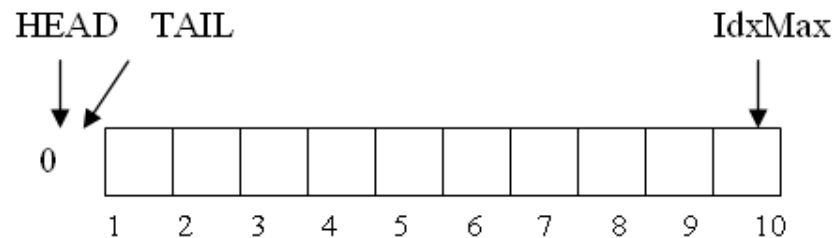
REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF I

- Jika Queue tidak kosong: TAIL adalah indeks elemen terakhir, HEAD selalu diset = 1
- Jika Queue kosong, maka HEAD = 0 dan TAIL = 0
- Ilustrasi Queue tidak kosong dengan 5 elemen



- Ilustrasi Queue kosong:



REPRESENTASI QUEUE DENGAN ARRAY

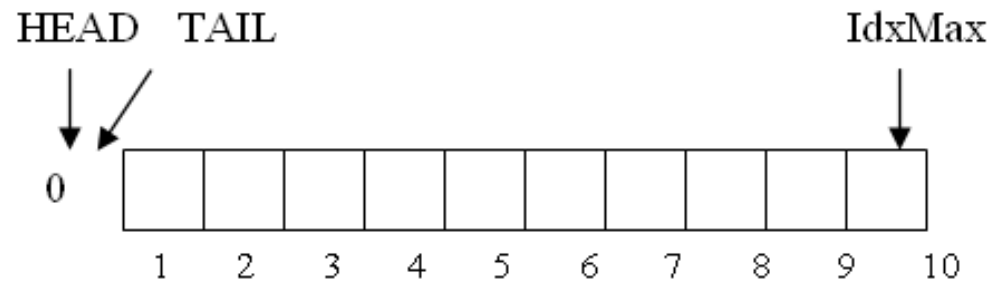
ALTERNATIF I

- Algoritma penambahan elemen:
 - ✓ Jika masih ada tempat, “majukan” TAIL
 - ✓ Kasus khusus untuk Queue kosong: karena HEAD dan TAIL harus diset nilainya menjadi 1
- Algoritma paling sederhana dan “naif” untuk penghapusan elemen
 - ✓ Jika Queue tidak kosong: ambil nilai elemen HEAD, geser semua elemen mulai dari $HEAD + 1$ s.d. TAIL (jika ada), kemudian TAIL “mundur”
 - ✓ Kasus khusus untuk Queue dengan keadaan awal berelemen 1, yaitu menyesuaikan HEAD dan TAIL dengan DEFINISI
- Algoritma ini mencerminkan pergeseran orang yang sedang mengantri di dunia nyata, tapi tidak efisien

REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF II

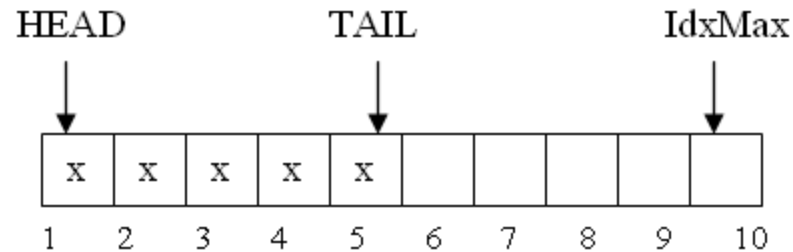
- Tabel dengan representasi HEAD dan TAIL, HEAD “bergerak” ketika sebuah elemen dihapus
- Jika Queue kosong, maka $HEAD = 0$ dan $TAIL = 0$
- Ilustrasi Queue kosong:



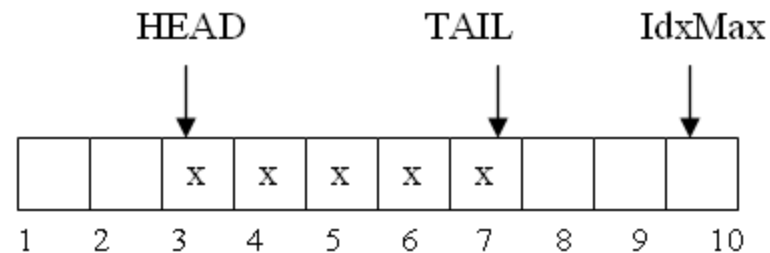
REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF II

- Ilustrasi Queue tidak kosong, dengan 5 elemen, kemungkinan pertama HEAD “sedang” berada di posisi awal:



- Ilustrasi Queue tidak kosong, dengan 5 elemen, kemungkinan lain HEAD tidak berada di posisi awal (akibat algoritma penghapusan)



REPRESENTASI QUEUE DENGAN ARRAY

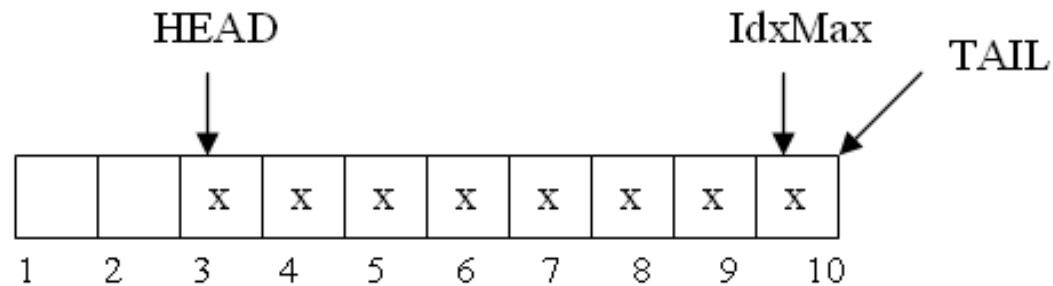
ALTERNATIF II

- Algoritma penambahan elemen sama dengan alternatif I, kecuali pada saat “penuh semu” (lihat slide berikutnya)
- Algoritma penghapusan elemen:
 - Jika Queue tidak kosong: ambil nilai elemen HEAD, kemudian HEAD “maju”
 - Kasus khusus untuk Queue dengan keadaan awal berelemen 1, yaitu menyesuaikan HEAD dan TAIL dengan DEFINISI.
- Algoritma ini TIDAK mencerminkan pergeseran orang yang sedang mengantri di dunia nyata, tapi efisien

REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF II

- Keadaan Queue penuh tetapi “semu” sebagai berikut:



- Harus dilakukan aksi menggeser elemen untuk menciptakan ruangan kosong
- Pergeseran hanya dilakukan jika dan hanya jika TAIL sudah mencapai IdxMax

REPRESENTASI QUEUE DENGAN ARRAY

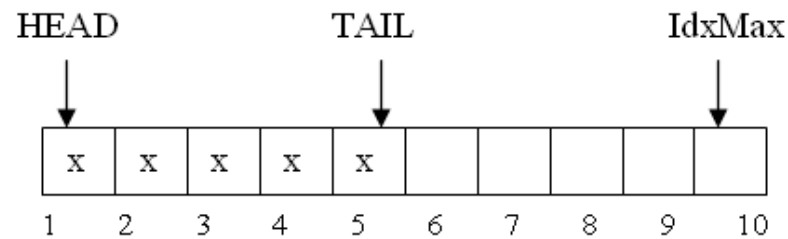
ALTERNATIF III

- Tabel dengan representasi HEAD dan TAIL yang “berputar” mengelilingi indeks tabel dari awal sampai akhir, kemudian kembali ke awal
- Jika Queue kosong, maka $HEAD = 0$ dan $TAIL = 0$
- Representasi ini memungkinkan tidak perlu lagi ada pergeseran yang harus dilakukan seperti pada alternatif II pada saat penambahan elemen

REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF III

- Ilustrasi Queue tidak kosong, dengan 5 elemen, dengan HEAD “sedang” berada di posisi awal



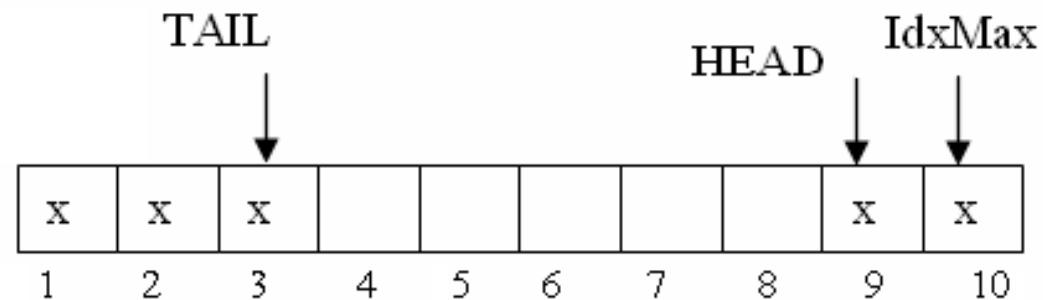
- Ilustrasi Queue tidak kosong, dengan 5 elemen, dengan HEAD tidak berada di posisi awal, tetapi masih “lebih kecil” atau “sebelum” TAIL



REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF III

- Ilustrasi Queue tidak kosong, dengan 5 elemen, HEAD tidak berada di posisi awal, tetapi “lebih besar” atau “sesudah” TAIL (akibat penghapusan/penambahan)



REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF III

Algoritma penambahan elemen:

- Jika masih ada tempat, “majukan” TAIL
- Jika TAIL belum mencapai $IdxMax$, maka algoritma penambahan elemen sama dengan alternatif II
- Jika TAIL sudah mencapai $IdxMax$, maka suksesor dari $IdxMax$ adalah 1 sehingga TAIL yang baru adalah 1
- Kasus khusus untuk Queue kosong karena nilai HEAD dan TAIL harus diset menjadi 1

REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF III

- Algoritma ini efisien karena tidak perlu pergeseran, dan seringkali strategi pemakaian tabel semacam ini disebut sebagai “circular buffer”, di mana tabel penyimpan elemen dianggap sebagai “buffer”
- Salah satu variasi dari representasi pada alternatif III:
 - Menggantikan representasi TAIL dengan COUNT (banyaknya elemen Queue)

REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF III

```
#define Nil 0
#define MaxEl 5

typedef int infotype, address;
typedef struct TEltList{
    infotype T[MaxEl + 1];
    address HEAD;
    address TAIL;
} Queue;

#define Head(Q) (Q).HEAD
#define Tail(Q) (Q).TAIL
#define InfoHead(Q) (Q).T[(Q).HEAD]
#define InfoTail(Q) (Q).T[(Q).TAIL]
```

REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF III

```
void CreateEmpty (Queue *Q) {  
    Head(*Q) = Tail(*Q) = Nil;  
}
```

```
bool isEmpty (Queue Q) {  
    return ((Head(Q) == Nil) && (Tail(Q) == Nil));  
}
```

```
bool isFull (Queue Q) {  
    return (NbElmt(Q) == MaxEl);  
}
```

REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF III

```
int NbElmt (Queue Q) {  
    if (isEmpty(Q))  
        return 0;  
    else {  
        if (Head(Q) <= Tail(Q))  
            return (Tail(Q) - Head(Q) + 1);  
        else  
            return (MaxEl - Head(Q) + Tail(Q) + 1);  
    }  
}
```


REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF III

```
void Add(Queue *Q, infotype x) {
    if (!isFull(*Q)) {
        if (isEmpty(*Q))
            Head(*Q) = Tail(*Q) = 1;
        else {
            if (Tail(*Q) == MaxEl)
                Tail(*Q) = 1;
            else
                Tail(*Q)++;

            InfoTail(*Q) = x;
        }
    } else
        cout << "Queue Penuh" << endl;
}
```

REPRESENTASI QUEUE DENGAN ARRAY

ALTERNATIF III

```
void Del (Queue *Q, infotype *hapus) {  
    *hapus = InfoHead(*Q);  
  
    if (Tail(*Q) == Head(*Q))  
        Head(*Q) = Tail(*Q) = Nil;  
    else {  
        if (Head(*Q) == MaxEl)  
            Head(*Q) = 1;  
        else  
            Head(*Q)++;  
    }  
}
```

```

int main() {
    Queue DataAntrian;
    infotype hapus, i;

    CreateEmpty(&DataAntrian);

    cout << "Add(2) " << endl; Add(&DataAntrian, 2);
    cout << "Add(3) " << endl; Add(&DataAntrian, 3);
    .....
    .....
    cout << "Del() "; Del(&DataAntrian, &hapus);
    cout << "Hapus " << hapus << endl;
    cout << "Isi Queue = ";

    while (Head(DataAntrian) != Nil) {
        cout << InfoHead(DataAntrian) << endl;

        Del(&DataAntrian, &hapus);
    }
}

```

IMPLEMENTASI STACK & QUEUE DENGAN LIST

- **STACK**
 - Proses push terjadi di posisi paling depan dari list, atau menggunakan fungsi InsertFirst
 - Proses pop terjadi di posisi paling depan dari list, atau menggunakan fungsi DeleteFirst
- **QUEUE**
 - Proses add terjadi di posisi paling belakang dari list, atau menggunakan fungsi InsertLast
 - Proses del terjadi di posisi paling depan dari list, atau menggunakan fungsi DeleteFirst

TERIMA KASIH